

Exam Introduction to Databases – 2nd session
3 September 2020

- The final exam contains 5 problems and is scheduled for 3 hours.
- Please verify if your question booklet consists of this cover page, 5 pages of questions, and 7 pages with supplementary lecture slides that are printed legibly. If this is not the case, contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. Supplementary lecture slides are provided at the end of this booklet.
- No electronic devices (such as notebooks, smartphones or smartwatches) are allowed.
- Usage of course material is not allowed.
- *Insufficiently detailed* answers are *void* (e.g.: you can't just answer "yes" or "no").
- Make sure to clearly mark your final answer to the questions; that is: do not only show the derivation, but make sure to state your conclusions in an unambiguous way.
- Take pride in your work: answer clearly and hand in a readable exam copy!
- For Problem 1 and 2 you can complete the SQL assignment using the available computers, which include postgres and DataGrip. The exam supervisor will collect solutions. All other questions must be answered on paper.

Problem 1: Entity Relational Model and Relational Design (5 points)

Question 1.2 and 1.3 may be answered using the class computer (paper is also possible).

Carefully read the following requirement analysis.

A publisher of travel guides is responsible for creating a book that contains various reviews of restaurants, hotels and sights. Your task is to create a database for the latest "Lonely planet Belgium" publication. This database provides access to employees, where for each employee we store the first and last name and date of birth. Within the database we store places, that is each place has a name (e.g. "Cathedral", "Restaurant de Rode Hoed", etc.), a type (e.g. hotel/restaurant/other) and an address. A place is uniquely identified by the combination of its name and address.

Each place is reviewed, where a review has a title, a description and a rating between 1 and 5 stars. Additionally, each review has zero or more tags (e.g. family-friendly/top-10-attraction/etc.). The review title, description, rating and tags can be edited by an employee and for each edit, we keep both the current and the previous versions including metadata. Metadata consists of the date, time and employee responsible for the edit. A review is uniquely identified by an identifier and the version number.

Finally, the database contains zero or more trips, where a trip is defined by a unique description (e.g. "One day in Antwerp") and an ordered path of different places (e.g. first visit "The Cathedral", then visit the "Restaurant de Rode Hoed", etc.).

1. Create an Entity-Relationship diagram that models as faithfully as possible the above description. Clearly indicate entities, relationships, primary keys, cardinalities and participation constraints.
2. Create the necessary create table statements to implement this ER-diagram as a relational database. Define all necessary schema-level constraints, such as primary keys, foreign keys and domain constraints. Active database instructions such as "On delete/update cascade/..." are not required.
3. Create a check to assert that each trip does not contain any repeating places (e.g. visit "Cathedral", "Restaurant X" then again "Cathedral").

Problem 2: SQL and Relational Algebra (5 points)

Question 2.2. may be answered using the class computer (paper is also possible). A minimal example schema will be made available.

The database schema for online purchases is:

Person(person_name, street, city, zip, country)

Company(company_name, street, city, zip, country)

Product(product_name, price, category, company_name)

Purchase(person_name, product_name, date)

You may assume that the underlined attributes indicate the keys of the relations. You may also assume that all joins you may need to express among the relations may be expressed as natural joins based on the matching attribute names among these relations.

Write the following queries in SQL:

- a. Find all the persons that did not buy a single product.
- b. Report the average price of products for each company in "Belgium".
- c. Report all pairs of persons who both have purchased three or more identical products.

Write the following queries in Relational Algebra:

- a. Find all pairs of person_name, company_name with persons who bought all the products of the respective company.
- b. Find all the distinct values of product_name that have been purchased by a person who lives in "Belgium", but which have been manufactured by a company that is not located in "Belgium".

Problem 3: Database design (4 points)

Consider the following database relations storing data of a system to autograde programming assignments of students. Students can submit solutions to exercises and those are automatically checked for correctness with some predefined input-output pairs. Every program is run multiple times on all inputs and the outputs are checked. There are several outcomes possible: the program may not compile, in which case all other outputs are recorded as NULL, or there could be an error in which case the error-attribute contains the type of error, and the output is NULL, or output is generated that is either correct or incorrect. Sometimes programs give different outputs when run multiple times on the same input; for instance if there are problems with memory management (dangling pointers).

A student can submit an assignment only once. For each submission the submission date and the number of lines of code (loc) is recorded.

Relation called "Overview"

ID	Name	Assignment	SubmissionDate	Late	Grade
0001	Jef Peeters	HW1	10/9/2020	No	12
0002	Ann Claessens	HW1	11/9/2020	Yes	10
0001	Jef Peeters	HW2	17/9/2020	No	12
0001	Jef Peeters	HW3	22/9/2020	Yes	8
0002	Ann Claessens	HW3	20/9/2020	No	14
...	...	...	...	...	...

Relation called "Tests"

ID	Assignment	Loc	Input	Repetition	Compile	RunTime Error	Correct
0001	HW1	121	1	1	Yes	No	Yes
0001	HW1	121	1	2	Yes	No	Yes
0001	HW1	121	2	1	Yes	SEGFault	n/a
0001	HW1	121	2	2	Yes	SEGFault	n/a
0001	HW2	52	1	1	Yes	SEGFault	n/a
0001	HW2	52	1	2	Yes	No	Yes
...	...	...	...	...	...	...	...

In these relations, the following rules hold:

1. The ID in table "overview" is unique for every student. *ID → student*
2. All students have the same deadlines for the assignments. This means that if two students submit the same assignment on the same date, either both are late, or both are on time.
3. In table "Tests", the code of a student is compiled only once. Hence, if the code did not compile for one input and repetition of that input, it fails for the other inputs as well.
4. In table "Tests", if the code did not compile, columns runtime error and correct are both "n/a" (not applicable). Otherwise RunTimeError contains No (no runtime error) and correct whether

2) ID, Assignment, → Grade, Late, SubmissionDate

4) ID, Assignment, Input, Repetition → RunTimeError, Correct, compile

3) ID, Assignment, Compile, Input, Repetition → Input, Repetition

or not the program produced the correct output, or if there is a runtime error it is listed in column RunTimeError and attribute Correct is "n/a".

5. The number of lines of code do not change between runs of the same submission.

ID, Ass → Loc
Question:

1. Which of the following functional and multi-valued dependencies hold? If a dependency does not hold, give a counter example:
 - a. Assignment, Input $\rightarrow$ Loc
 - b. Correct, Compile $\rightarrow$ Compile
2. List all candidate keys for both relations.
3. Are these two relations in BCNF? Give arguments to support your answer! If the database is not in BCNF, decompose the relations to reach a lossless BCNF-decomposition.

Problem 5: NoSQL (2 points)

What does the CAP-theorem stand for? Explain what its claims are in your own words. Illustrate with small examples where necessary.

Answer Problem 5:

student name:

Defining Foreign Keys

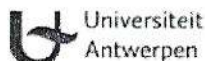
```
CREATE TABLE Orders
```

```
(  
  O_Id int NOT NULL,  
  OrderNo int NOT NULL,  
  P_Id int,  
  PRIMARY KEY (O_Id),  
  FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)  
)
```

Does not have to be the same attribute name. But number and types need to match

Could also be multiple attributes

Enforces that every Orders.P_Id occurs in the Persons table in the column P_Id.



Assertions

(not supported in Oracle/Postgres, uses CREATE TRIGGER instead)

MovieExec(name, address, certN, netWorth)
Studio(name, address, presCertN)

- No one may be the president of a studio, unless this person has a net worth of at least \$1M :

```
CREATE ASSERTION RichPres CHECK  
(NOT EXISTS (  
  SELECT * FROM MovieExec, Studio  
  WHERE certN = presCertN AND netWorth < 1000000));
```

- Every executive name has only one address :

```
CREATE ASSERTION FunctionalDependency CHECK  
(NOT EXISTS (  
  SELECT * FROM MovieExec mv1, MovieExec mv2  
  WHERE mv1.name = mv2.name AND mv1.address <> mv2.address));
```



Triggers in SQL

General syntax:

```
CREATE TRIGGER <trigger_name>
{BEFORE|AFTER} {{INSERT|DELETE|UPDATE} [OF <row_list>]}* ON <table_name>
[REFERENCING [NEW ROW|TABLE AS <new_name>] [OLD ROW|TABLE AS <old_name>]]
[FOR EACH ROW|STATEMENT [WHEN (<trigger_condition>)]]
BEGIN
  <trigger_body>
END;
```

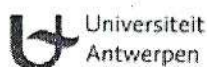
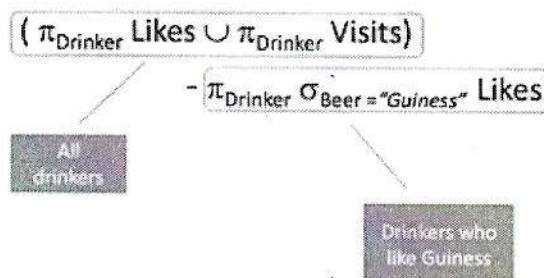


* SQL'99 allows both ROW and TABLE/STATEMENT level triggers.
Oracle allows only ROW level, hence the keyword ROW is omitted.

Examples

Give all drinkers who do not like "Guinness"

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)



Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who visit a bar that serves "Guinness"

$$\pi_{\text{Drinker}} \sigma_{\text{GPub=Pub}} (\text{Visits} \times (\rho_{\text{GPub/Pub}} \sigma_{\text{Beer="Guinness"}} \text{Serves}))$$

Give all drinkers who visit a bar that serves a beer they like

$$\begin{aligned} & \pi_{\text{Drinker}} \sigma_{\text{VP=SP} \wedge \text{SB=Beer} \wedge \text{Drinker=VD}} \\ & (\text{Likes} \times \rho_{\text{SP/Pub, SB/Beer}} \text{Serves} \times \rho_{\text{VD/Drinker, VP/Pub}} \text{Visits}) \\ = & \\ & \pi_{\text{Drinker}} \sigma_{\text{SB=Beer} \wedge \text{Drinker=VD}} [\text{Likes} \times \\ & \sigma_{\text{VP=SP}} (\rho_{\text{SP/Pub, SB/Beer}} \text{Serves} \times \rho_{\text{VD/Drinker, VP/Pub}} \text{Visits})] \end{aligned}$$

Keys & Functional Dependencies

A set of one or more attributes $\{A_1, A_2, \dots, A_n\}$ is called a **key** for a relation R if:

- (1) $\{A_1, A_2, \dots, A_n\}$ functionally determines *all attributes* of R .
- (2) No proper subset of $\{A_1, A_2, \dots, A_n\}$ functionally determines all attributes of R .

→ In other words, a key must be a **minimal set of attributes** for which property (1) holds.

A set of attributes $\{A_1, A_2, \dots, A_n\}$ that contains a key of relation R is called a **superkey** of R .

→ Every key is also a superkey.

A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:

If $\{B_1, B_2, \dots, B_m\} \subseteq \{A_1, A_2, \dots, A_n\}$,
then $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ ($\rightarrow$ "trivial FDs")

2. Augmentation:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$,
then $A_1, A_2, \dots, A_n, C_1, C_2, \dots, C_k \rightarrow B_1, B_2, \dots, B_m, C_1, C_2, \dots, C_k$,
for any set of attributes $C_1, C_2, \dots, C_k$ ($\rightarrow$ "non-trivial FDs")

3. Transitivity:

If $A_1, A_2, \dots, A_n \rightarrow B_1, B_2, \dots, B_m$ and $B_1, B_2, \dots, B_m \rightarrow C_1, C_2, \dots, C_k$,
then $A_1, A_2, \dots, A_n \rightarrow C_1, C_2, \dots, C_k$ ($\rightarrow$ "closure algorithm")

Normal Forms of Relations

In the following, let X be a set of attributes and Y be a single attribute of a relation R , and let F be a given set of FD's on R .

R is in Third Normal Form (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R .

R is in Boyce-Codd Normal Form (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R .

Algorithm for Computing a Decomposition in BCNF (not dependency-preserving!)

Given a relation R with attributes A , a set of keys K , and a set of FD's F .

If there is a non-trivial FD $X \rightarrow Y$, where X is not a superkey of R , then decompose R into:

- $S = \pi_{X \rightarrow F}(R)$ with the new key X ;
- $T = \pi_{X \cup (A - X) \rightarrow F}(R)$ by taking over all remaining keys from R .

Then recursively decompose S and T into BCNF using the remaining (non-violated) FD's in F .

Algorithm for Computing a Dependency-Preserving Decomposition in 3NF

Given a relation R with attributes A , a set of keys K , and a *minimal basis* of FD's F .

Decompose R into:

- $R_1 = \pi_{K'}(R)$ where K' is a key in K which becomes key of R_1 ;
(if there are multiple keys in K , choose one)
- for each $X \rightarrow Y$ in F ,
the projection $R_{i+1} = \pi_{X \cup Y}(R)$ forms a new relation in the decomposition with key X .

Multivalued Dependency

A **multivalued dependency** (MVD), denoted as

$$A_1, A_2, \dots, A_n \twoheadrightarrow B_1, B_2, \dots, B_m$$

holds for a relation R if, for each pair of tuples t, u of R , which agree on all the A 's, there is a tuple v in R that also agrees

- with both t and u on the A 's;
- with t on the B 's;
- with u on *all other attributes* of R not among the A 's and B 's.

Example:

	starName A's	city,street B's	title,year others
t	a_1	b_1	c_1
	Ford	Malibu, Maple Str.	Star Wars, 1977
v	a_1	b_1	c_2
	Ford	Malibu, Maple Str.	Star Wars, 1980
u	a_1	b_2	c_2
	Ford	Hollywood, Mulh. Dr.	Star Wars, 1980

The Two-Phase Locking Protocol

- This protocol ensures conflict-serializable schedules.
- Phase 1: Growing Phase
 - Transaction may obtain locks
 - Transaction may not release locks
- Phase 2: Shrinking Phase
 - Transaction may release locks
 - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).

Timestamp-Based Protocols (Cont.)

- The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.
- Suppose a transaction T_i issues a **read**(Q)
 1. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten.
 - Hence, the **read** operation is rejected, and T_i is rolled back.
 2. If $TS(T_i) \geq W\text{-timestamp}(Q)$, then the **read** operation is executed, and $R\text{-timestamp}(Q)$ is set to $\max(R\text{-timestamp}(Q), TS(T_i))$.



Thomas' Write Rule

- Modified version of the timestamp-ordering protocol in which obsolete **write** operations may be ignored under certain circumstances.
- When T_i attempts to write data item Q , if $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of $\{Q\}$.
 - Rather than rolling back T_i as the timestamp ordering protocol would have done, this **{write}** operation can be ignored.
- Otherwise this protocol is the same as the timestamp ordering protocol.
- Thomas' Write Rule allows greater potential concurrency.

